

Chip Development Best Practices

This is:

- Collective wisdom on coding & engineering practices used in Chip development.
- Mainly for people working with Chip, but some of the points apply to C++ development in general too.

This is not:

- A list of Chip-specific issues to consider when writing code. For that, see [Chip WHY](#).

PRs and Code Reviews

1. Read [Pull Request Procedures](#).
2. Always determine which version branch (3.1, 3.2, etc.) to begin making your changes to. Double check with Julie/John/Joe before getting started. Nothing is worse than having to do additional work because you or someone else made changes to the wrong version.
3. Prefer more PRs to fewer when making unrelated changes in multiple places.
4. Make logical commits (one commit for one functional change).
5. Commit messages must begin with the name(s) of the affected component(s). For example "CmeMD: fix exchange sequence reversal" not "fix bug in exchange sequence".
6. Don't commit formatting & non-formatting changes together in the same PR, and certainly not in a single commit.
7. Review your changes in Github before others do. You can review the branch diff before creating a PR, or create a draft PR (more on that below). Very often people catch errors and typos when looking at the diff that they didn't see while coding.
8. Write a PR description that explains why you're proposing these changes. What problem are you addressing, what is the behavior before and after, etc. Think "why" not just "what."
9. In the PR description, mention the issue it is meant to fix (if there is one) in the description, e.g. "Resolves [#6740](#)". If your PR is against the default branch, the issue will be automatically closed when the PR is merged. Even if it is not, though, the PR will show up in the issue as having "mentioned" it, which is useful for manually cleaning up issues later if necessary.
10. When making changes after PR comments, unless the PR is very small it's best to put them in new incremental commits rather than squashing the old commits and force pushing. This makes it easier for the reviewer to see what changed from the last version. Once the review is approved, you may wish or be asked to squash your commits before it is merged.
11. Also see: [Git Tips](#)

Modern C++

1. See: <http://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>, [Effective Modern C++](#), [Modern C++ Design](#).
2. Use the c++ keyword "using" as opposed to the c style "typedef" when defining aliases. See <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#t43-prefer-using-over-typedef-for-defining-aliases>
3. Avoid using raw new and delete to dynamically allocate memory. See <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r11-avoid-calling-new-and-delete-explicitly>
 - a. If you need to dynamically allocate memory, prefer Unique Pointers. Chip has a `unique_ptr` helper library which you can see here: http://git.drwholdings.com/Chip/core/blob/release/Chip_3.2/src/Core/Core/Util/UniquePtr.h
 - i. Limit the exchange of ownership when working with a `unique_ptr`. Whatever object allocates the memory, should ideally hold onto it for the entire lifetime. See <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#i11-never-transfer-ownership-by-a-raw-pointer-t-or-reference-t>
 - ii. You can safely pass the `unique_ptr`'s data (raw pointer) into functions, guaranteeing the lifetime of the memory via the owner of the `unique_ptr`
 - iii. Avoid storing a reference to the `unique_ptr`'s data anywhere, and NEVER store it somewhere that may outlive the `unique_ptr`.

Creation of unique pointer and passing as argument to function

```
// lifetime of a MarketDataService instance is from Core::App initialization stage till app
shutdown.
class MarketDataServiceCommon : public TradingBusiness::MarketDataService
{
    ...
protected:
    // After a handler for a requested contract finishes loading, unique pointer to
    that handler is passed to MDSERVICE instance
    // to express a change in ownership. MDSERVICE instance is now responsible for
    managing lifetime of that handler.
    void NotifyLoadComplete(const TradingBusiness::MarketDataContract & contract, std::
unique_ptr<MarketDataHandlerCommon> handler)
    {
        _handlers[handler->GetContractSpec()] = std::move(handler);
        ...
    }

private:
    // Store 1 handler of market data per contract
    // When this instance of MDSERVICE is destroyed, data owned by the unique pointers
    are automatically
    // deallocated, helping us avoid forgetting to call free if raw pointers are used
    instead of unique pointers.
    std::map<TradingBusiness::MarketDataContract, std::unique_ptr<MarketDataHandlerCommon>>
_handlers;
};

class MarketDataHandlerLoader
{
    ...
protected:
    void NotifyComplete(std::unique_ptr<MarketDataHandlerCommon> handler)
    {
        ...
        if (/* some condition */)
            _service->NotifyLoadComplete(GetContractSpec(), std::move(handler));
    }

private:
    MarketDataServiceCommon * _service;
};

// Where MD handler unique pointer is first constructed
template<typename Adapter>
class KrxMarketDataServiceLoader : public MarketData::MarketDataHandlerLoader
{
    ...
private:
    void OnVenueInstrument(const KrxInstrument* instrument, Core::ErrorCode error)
    {
        if (error)
            ...
        else if (/* some condition */)
        {
            auto handler = Core::MakeUnique<InstrumentHandlerType>({
                GetService<MarketDataServiceType>(),
                _adapter,
                GetContractSpec(),
                instrument
            });
            NotifyComplete(std::move(handler));
        }
        else
            ...
    }
};
```

Accessing data held by unique pointer

```
// lifetime of a MarketDataService instance is from Core::App initialization stage till app
shutdown.
class MarketDataServiceCommon : public TradingBusiness::MarketDataService
{
public:
    TradingBusiness::MarketDataHandler * Subscribe(const TradingBusiness::
MarketDataContract & contract) final
    {
        ...
        auto handler_i = _handlers.find(contract);
        if(handler_i != _handlers.end())
            return handler_i->second.get(); // returns raw pointer to whoever
wants access to resources whose lifetime we manage
    }

private:
    std::map<TradingBusiness::MarketDataContract, std::unique_ptr<MarketDataHandlerCommon>>
_handlers;
};

// Instances of this class are created by application whenever it needs access to market
data.
// Lifetime of this instance is thus a subset of lifetime of MDSERVICE instance
class MarketDataSubscriber
{
public:
    MarketDataSubscriber(MarketDataService * service, const MarketDataContract &
contract, const MarketDataParams & params, const StatusCallback & statusCb)
        : _service(service)
        // Obtain raw pointer to MD handler owned by service instance.
        // Since this instance of MDSubscriber never lives longer than MDSERVICE,
        // we can be sure that _handler won't be pointing to garbage due to early
destruction of MDSERVICE.
        , _handler(_service->Subscribe(contract))

    { ... }

private:
    Core::PointerFlag<MarketDataService> _service;
    MarketDataHandler * _handler;
};
```

- b. There are times when it's acceptable to allocate memory on the heap using **new**. Expect a healthy amount of skepticism when attempting to use raw **new/deletes**, and follow these rules:
 - i. Match any **new** with an opposite and correlated **delete**
 - ii. Use Valgrind to detect potential leaks
 - iii. Own the allocated memory in a single place. You will have a bad time if its lifetime is managed in more than one place.
 - iv. Keep the dynamically allocated memory contained locally to the object that created it. If you start allocating memory on the heap, then passing it around, you're going to have a bad time.
4. Take advantage of **auto** when it improves readability. In non-template code where the type of a variable is far removed from its declaration, maybe don't use **auto**.
5. Avoid C style strings. If you have to use them, be aware of common dangers.
 - a. Copying into a buffer of the wrong size is easy to do and will break a lot of things.
 - b. Missing terminating character when treating it like a c++ style string can cause logging and output issues. Likewise having a null character in the middle.
 - c. Consider whether **std::string_view** is a better choice, or for logging use **Core::LogNoCopy** (which has a user-defined literal syntax: "hello"_log).

Other Best Practices

- for (**auto&** element : collection) to avoid unnecessary copies.
 - if you're intending to grab a pointer, use **auto * obj**. It allows future readers to quickly confirm it is not an accidental copy.
- Don't define a destructor if you don't need to. The most-base class which has virtual functions should have a virtual destructor. Defining a destructor in the .cpp file lets you reduce #includes if you hold a smart pointer in a class and the referent type can be forward declared (hence the compiler-provided destructor would fail to compile).

- Make move special member functions (default constructor, copy/assignment/move, destructor) noexcept.
- If a constructor can be called with a single argument, make it `explicit`.
- If a constructor takes multiple arguments, consider wrapping them in a struct - especially if the arguments will be passed through to other constructors.
- Avoid default arguments on abstract functions. Wrap them in a template method (aka non-virtual-interface).
- If returning any empty container, etc - just use `{}`.
- Use runtime analysis tools, especially when developing new code. UBSan can be used all the time in development. See [ASan, UBSan and TSan](#)
- Null pointers are rare in our code base, and where a class/function takes something that is truly optional/nullable, the argument should be named appropriately (eg, `optionalContract`)
- Don't hold C++ references to external objects. Knowing that something is not null is much less important than knowing 1) that it's external memory that can change outside of your control, and 2) that it's lifetime must be maintained outside your class. Specifically:
 - When reading code, you can't distinguish an external reference member from your own members without referring to the declaration. Reading `_member -> F()` avoids that.
 - When reading the call site, there's no indication that the callee might retain a reference to that memory, and thus no indication that the caller needs to ensure it outlives the callee. Seeing `&argument` makes that explicit.

Optimization

1. Be conscious of padding and data member ordering when designing simple structures. Its possible to cause an object to be larger than is required: <https://godbolt.org/z/jxv6hr>
 - a. A good rule of thumb is to list members in order from biggest to smallest.
 - b. This becomes less of an issue for larger structures.
 - c. When modifying a small structure, changes that add members without increasing the size will be less scrutinized than a change that doubles the size of an object just to squeeze in a byte of information. Consider this when modifying particularly small structs.
 - d. When ordering members place 'fast path' data near the start of the structure and contiguously. If your object has multiple fast paths (for example, a receive and send path) place members for these paths contiguously.
 - e. Use `CORE_ASSERT_SIZEOF` in `Core/Util/Assert.h` to inspect (and enforce if desired) the size of a struct. For example, you can do `CORE_ASSERT_SIZEOF(ThisClass, 0)` to print out the current size at compile time.
 - f. If a member has padding at the end (e.g. you have a member which is a struct of `uint32_t` and `uint8_t`), use `[[no_unique_address]]` to make that slack space available for subsequent fields.
2. BigO time complexity is an important concept to understand and should be considered when developing algorithms, but know that it isn't the sole indicator of performance. I think some internal example demonstrating some of the concepts here would be good <http://umich.edu/~eecs381/handouts/FillErUp.pdf>
 - a. Consider spatial locality and cache loading when deciding which data structures to use. Scanning a small vector for an object is faster than finding it in an equally as small `unordered_map`.
3. If you know in advance the size of any container with a `reserve()` function like `std::vector`, `unordered_map`, `unordered_set`, `string`: call `reserve` before filling to avoid unnecessary and oversized allocations.
4. Free `unique_ptr`'s before reuse, this is usually faster as the internal allocation that happens can reuse the memory that was just freed rather than finding a new empty block.
5. Always prefer doing as much work up front when working with performance sensitive code. Preallocate, and pre-fill as much information as possible before a latency-sensitive action occurs.
 - a. When using vectors, either `reserve()` or use `Core::FixedVector` (the latter is required for immovable objects).
6. Know your fast path(s) and make sure they stay fast.
 - a. Identify early on. Eliminate as many allocations as possible, and precalculate complex status conditions if needed. Ask someone if it isn't obvious
 - b. Usually in order adapters, receiving fills is the high-priority fast path, leg fills less so. Sending `submit/revise/cancel` are also fast path.
 - c. If you find yourself needing a dynamically allocated string or vector in your fast path, consider keeping one in your class pre-allocated to use as 'scratch' space.
7. Conversely, know when you are writing a path that will never need to be latency-sensitive and prioritize clean, readable code instead.

Headers

1. In general, include what you introduce or require for the module you're writing. Don't include things required by your module's dependencies.
2. Clean up your `#includes` before opening a pull request.
3. Be aware of the performance overhead associated with calling a function https://en.wikipedia.org/wiki/Call_stack. Most compilers are smart enough to inline short simple functions to avoid this overhead. If we want a function to be inlined, write the function's definition in the header. If the compiler's optimizer does not inline as we want it to, there is the `'always_inline'` attribute (not really used in Chip: in most cases the compiler's optimizer and our optimization needs agree).
 - a. Note, the keyword `'inline'` used to be a recommendation to the compiler to do inline expansion of a function. In modern compilers this is no longer the case, and `'inline'` now is meant to tell the linker that this function's definition can show up in multiple translation units. See <https://stackoverflow.com/a/1759575/4130137> for a clear explanation of this.
 - b. If something is not in the hot path and it's more than a couple lines of code, default to defining it in a .cpp file.
 - c. Be aware of where inline is implied, eg, member functions defined within a class definition and `constexpr` are implicitly inline. Even template functions imply one of the benefits of inline.
4. Know the difference between internal and external linkage, and how the keywords `static`, `inline`, and `constexpr` affect linkage in c++11.

Namespaces

1. Avoid using nested namespaces. Always wrap your code in at least one specific namespace to avoid name collisions. **ToDo: Don't give a blanket negative statement, show where we do this**

Functions

1. Pay close attention to the types you pass into functions. Prefer `const&` to passing by value.
 - a. If a trivial object takes up 16 bytes or less (e.g. two pointers), pass by value, otherwise by reference. For the purpose of this point, a 'trivial' object is one without user defined copy constructor or destructor. Specifically, even though `std::unique_ptr` is 8 bytes, it is still more efficient to pass them by reference than value (likewise for `shared_ptr`).
 - b. When passing parameters to a function, attempt to bundle as much as you can into a struct. Avoid passing in a bunch of loose values. It can get confusing to call a function with many (4+) arguments.
 - i. In this example `MidOffice::ExchangeAccountEntry` bundles a number of parameters and is passed as a single object into the function
 1. <https://git.drwholdings.com/Chip/core/blob/3b66aaa4358582de50da52899c99daa5a9d3af22/src/Exchanges/CboeC1/CboeC1/OrderAdapter/CboeC1OrderSession.h#L81>
 - ii. If the parameters needed for the function change, bundling the input often reduces the amount of code that needs to change.
2. Appropriately mark functions as `const`, `override` or `final`. This guarantees that no one accidentally abuses a function, or uses code in unintended ways. This can be an easy thing to miss, so consider doing a specific check for this before opening a PR. You can mark the entire class `final` if appropriate, which has the effect of making all overrides final as well as any compiler-provided destructor etc.

Classes and class design

1. Minimize the amount of necessary state whenever possible.
 - a. State maintenance can lead to some very tricky bugs
 - b. Use existing objects and containers to keep state whenever you can
 - c. In the code linked below, I use the string `_riskRoot` to determine if this method has been called before. Previous versions of this code stored a `_hasContractBeenDiscovered` member, which was entirely redundant and unnecessary
 - i. https://git.drwholdings.com/Chip/core/blob/release/Chip_3.2/src/Exchanges/CboeC1/CboeC1/MassQuoting/CboeC1Group.cpp#L52
 - d. Related to minimizing state, also minimize the initialization of a class to only a single phase (in the constructor), where possible. Multi-phase initialization of a class is more error-prone.
2. Be very deliberate when defining public/protected/private members of a class
 - a. Avoid making any member variable public, prefer public accessors.
 - b. Only make functions public that users will need, hiding everything else away privately. When in doubt, define something as private and promote if needed.
 - c. If you need to test private functionality, first consider whether refactoring that logic into a new object or free function would be more appropriate. Other options are using the attorney-client pattern (https://en.wikibooks.org/wiki/More_C%2B%2B_Idioms/Friendship_and_the_Attorney-Client) with a friend test fixture class, creating a mock class which redeclares these functions as public, and/or making the function public with a comment that it is intended for testing only. **ToDo: Examples from Camus/toy code samples**
3. Nested classes should not be public. If a class definition is public it should be at namespace scope, not inside another class which prevents forward declaration. See <https://stackoverflow.com/questions/951234/forward-declaration-of-nested-types-classes-in-c>
4. Mark destructors `noexcept`. Even though the compiler implicitly makes destructors `noexcept`, if something in your class (e.g. base class or member variable) has a throwing destructor, it will make your class' destructor no longer implicitly `noexcept`. Marking `noexcept` is a best practice that prevents a throwing destructor from sneaking into the code.

Types

1. Use fixed width integer types <https://en.cppreference.com/w/cpp/types/integer>. This guarantees you get the exact amount of space you want. Don't over pay when a bits will do, and don't get caught overflowing when you need 64 bits.

Containers

1. `std::list`, `std::unordered_map`, `std::map`: references to elements are not invalidated on insertion; `std::map`, `std::list`: iterators are not invalidated on insertion
2. `DenseHashMap/Set`: this is google's `dense_hash_map`, which is an open addressing hash table with a quadratic probing collision resolution scheme. Generally-speaking, if you do not need your map entries to have a fixed location in memory, then the `DenseHashMap` is generally superior in performance to the `std::unordered_map`. The benefit of `DenseHashMap` is especially compelling with keys/values which are small and cheaply moveable. `DenseHashMap` has two common downsides. First, iterators can be invalidated on insertion/deletion. Second, it can automatically shrink the buckets array upon some deletions.
3. Understand intrusive lists?

Macros

1. Only use macros when there is no other way to achieve what you want.

Casting

1. Avoid implicit casting whenever possible. When changing types, attempt to be as explicit as possible to guarantee expected results.
2. Avoid using C-style casts (e.g. `(CastToType) objOfType;`), unless you are casting between primitive types (e.g. `int32` to `int64`).
3. C++ offers 4 casting operations. Understand the appropriate use of each
 - `static_cast`
 - Static cast attempts to convert an object of one type into another.
 1. Static cast can convert built-in types. Be warned that this can strip precision and you won't get any indicator that this is happening. This can also cause slicing if done to an object on the stack, or when passing by value to a function
 2. Static casting should be used to properly upcast an object, and in most cases should be used when down casting to guarantee type safety
 - `reinterpret_cast`
 - i. Converts any built-in type into any other, or any pointer type into another pointer type

- ii. This is primarily used to convert abstract buffers into usable data structures. Be careful that what you're reinterpreting is complete, and will give you a valid object.
- dynamic_cast
- Dynamic cast is primarily used to cast an object of one type, into another that is in a shared inheritance hierarchy. When downcasting a base class to a derived class, dynamic_cast will do a runtime check to verify that the object really is of the derived class. If we did a static_cast of Base to Derived where the object is not actually a Derived and then attempted to use the object as a Derived, we would get undefined behaviour.
1. Generally avoid dynamic_cast, because in almost all cases what you actually want to do is either static_cast, or a virtual function. This advice is directed at the people who are just trying to "do something" and think that dynamic cast might do what they want.
 2. If you are sure you should use a dynamic_cast in a particular situation then go ahead and use it. e.g. you're making a tradeoff so you can let your base class have a reduced interface with fewer mandatory virtual functions).
 3. Not that unlike the other casts, dynamic cast incurs additional performance overhead as it does runtime validation that a cast is possible
- const_cast
- i. strips the "const" qualifier of a variable. Using const cast should be avoided. If something is marked const, it's for a reason. If you find yourself needing const_cast, determine if there is a more suitable way to accomplish your goal.
 - ii. See the mutable specifier <https://en.cppreference.com/w/cpp/language/cv>

General engineering

1. Avoid code duplication, specifically within a file or folder. If you find yourself copy/pasting consider factoring something out into a common function
 - a. Note - It's possible to lean TOO heavily into this. Not every simple action needs to be moved into a separate function. It is frustrating to hop around a file trying to figure out what some 2 line helper function is doing.
2. Avoid global variables, they can be very error prone as it is unclear when they are read or written. If you need to define global constants, consider using `constexpr` functions. These have the advantage of not needing to store a global variable, while still allowing you to have a global constant defined in one location:
 - a. <https://git.drwholdings.com/Chip/core/blob/19c321494ea7e1225b16e8aba42d700ca0953772/src/Exchanges/Cme/Cme/ILink3/ILink3Client.cpp#L17>

Common coding patterns

1. Resource Acquisition is Initialization (RAII)
 - a. <https://en.cppreference.com/w/cpp/language/raii>
 - b. Do not begin constructing an object that can't possibly be finished. Don't create something that will forever exist in a damaged or down state. Avoid multiple-phase initialization if possible, that is, your object should ideally be in a valid, fully-constructed state upon its constructor being completed.
 - c. Check that you have all of the necessary resources and validate params before constructing an object
 - d. **ToDo: Link to an order session loader or something. We actually break rule 1.b sometimes, allowing sessions to be created that will forever live in the down state. I may add a note that it isn't always possible to enforce that rule**
 - e. **ToDo: Show how to do this when needed. COG / TcpAcceptor example?**
2. CRTP
 - a. https://en.wikipedia.org/wiki/Curiously_recurring_template_pattern
 - b. example: `core/src/Core/Core/Util/RingBuffer.h`, `RingBuffer` injects itself as a template argument into its base template class `SimpleRingBufferInterface`
 - c. most commonly used in our code for static polymorphism
3. SFINAE
 - a. Substitution Failure Is Not An Error - use template substitution failure to remove functions overload resolution set. Can be used to remove function definitions that are inapplicable for the given template parameters, such as having one version of the function for if the provided type has function `ToString` and another if it does not. Search the file linked below for "SFINAE" to see examples of how it is used.
 - b. <https://git.drwholdings.com/Chip/core/blob/aa1e3bcabc488943898b8e6e75b9dc57975857ad/src/ExchangeFramework/OrderAdapter/OrderAdapter/OrderActionImpl.h#L110>
 - c. <https://en.cppreference.com/w/cpp/language/sfinae>
 - d. From c++20 onwards, we should use Concepts instead of SFINAE: <https://en.cppreference.com/w/cpp/language/constraints>. Concepts can do everything we used to do with SFINAE

Common bugs

1. See [CHIP WHY](#).
2. Always finish updating an object's local state before invoking a callback or notifying any handlers. It's dangerous to cede control until the local object has done all it needs
3. If your base class is destructed polymorphically, remember to add a virtual destructor. Always check for polymorphic destruction if your base class is used polymorphically in any way. (<https://git.drwholdings.com/Chip/core/pull/2113>)
4. Be careful when using user-defined class comparators in `std::set` and `std::map`, it's easy to write them wrongly (<https://git.drwholdings.com/Chip/core/pull/2666>, <https://git.drwholdings.com/Chip/core/pull/1595>)
5. Do not use objects which have been moved except in the ways which are guaranteed to work. For STL containers, `clear()` is perhaps the only sensible operation. See <https://git.drwholdings.com/Chip/core/pull/4643>
6. Do not call pure virtual functions in constructor. See <https://git.drwholdings.com/Chip/core/pull/4370>
7. Be careful when storing pointers to objects which reside in a `std::vector` and `dense_hash_map/set`. Those pointers will be invalidated on `resize`. See <https://git.drwholdings.com/Chip/core/pull/3144>

Misc (to be organized)

1. Define non-member functions exclusively used by a single object in the .cpp file.
2. Take advantage of static analysis whenever possible. Let the compiler catch potential bugs before ever running the executable.
 - a. If you must pass a cstyle array to a function, enforce the size of the array via the functions definition: <https://godbolt.org/z/vjj4eq>
 - b. Use `static_assert` to verify information known at compile time: <https://git.drwholdings.com/Chip/core/blob/19c321494ea7e1225b16e8aba42d700ca0953772/src/Exchanges/Cme/Cme/Link3/Link3Client.cpp#L519>
 - c. Use `enum class` to define possible values when there is a small, well defined set.
 - d. Don't use ``default`` in switches to catch unused values; switch only over the defined values, returning from each case, and then add "default" code at the end. This lets us have our default case while letting the compiler remind us to add cases for any newly defined values.
 - e. Consider using `Clang-Tidy`.
3. Define static member functions when a member function doesn't rely on the object's members
 - a. In the code linked below, all instances of `FillItem` have the same name. This method doesn't care about the specific instantiation of `FillItem`, and can be made static
 - b. https://git.drwholdings.com/Chip/core/blob/release/Chip_3.2/src/Exchanges/Cme/Cme/Link3/Link3Messages.h#L1417
4. Any "Register" (storage of a reference to some other object) should have an opposite "Unregister" (removal of the reference)
 - a. If you're storing a callback or reference to an object, it's important that you have a way to remove that callback or reference in the case that the underlying object is destroyed.
 - i. In the code linked below note that the `CboeC1Group` registers itself with the `CboeC1MassQuoteAdapter` in the group constructor. The `MassQuoteAdapter` stores a pointer to this newly created group, which it doesn't own, and doesn't necessarily share a lifetime with (The group can be destroyed independently of the adapter)
 1. https://git.drwholdings.com/Chip/core/blob/release/Chip_3.2/src/Exchanges/CboeC1/CboeC1/MassQuoting/CboeC1Group.cpp#L41
 2. https://git.drwholdings.com/Chip/core/blob/release/Chip_3.2/src/Exchanges/CboeC1/CboeC1/MassQuoting/CboeC1MassQuoteAdapter.cpp#L410
 - ii. See that later in the group's destructor, it unregisters itself from the adapter, ensuring that the adapter can never accidentally interact with it again
 - b. Pay close attention anytime you find yourself storing memory owned by something else, including pointers or callbacks. Determine the relative lifetimes, and when in doubt, just remove the reference whenever the memory becomes unsafe.
5. Make liberal use of `CORE_ASSERT` to guarantee state and prevent undefined behavior. Asserts keep code clean, and help debugging
 - a. `CORE_ASSERT` is like `std::assert` but allows you to pass in a message which is logged when the assert triggers. This is helpful for debugging. The message is actually a format string like `printf()` - include the values involved in the assert so we know what they were if it trips.
6. Section on Core/Utils that should be known about. Possibly bake this into the above note?
 - a. All of the good stuff in `Conversions.h`. Maybe tough to list it all here, maybe just link the file
 - b. `Core::Delegate` vs `std::function` **ToDo: explain**
 - c. `Range.h` for quickly returning container references
 - d. `StringUtils.h` is criminally underused
 - e. `Core::FixedVector`, `Core::SmallVector`, etc.
 - f. Type traits / TMP utils including tools for `SFINAE`
7. Take advantage of required services, and rely on base classes `RequiredServices`. Know why you're including a service, don't just copy and paste
8. Use appropriate logging macros for appropriate situations. Prefer lower severity when you're not sure. `ERROR` had better be serious and actionable. For more info on different log levels meanings, [see here](#).
9. OA/MDC constructors: where applicable (`Adapter.h/cpp`, `Session.h/cpp`, `Channel.h/cpp` etc.), the preferred listing of constructor arguments is `RequiredServices`, followed by any parameters. [See here for an example](#)